WEEK 4 PROGRESS REPORT

1. Introduction

During Week 4, work on CampusConnect focused on moving beyond answer generation by giving the assistant explicit, safe software capabilities. The aim was to define tools with clear contracts, allow the model to request them through function calling, and ensure that the application, rather than the model, decides whether each request may run.

Four tools were exposed to the Gemini model through function calling. They are backed by a local SQLite database containing synthetic timetable and support-request data, as agreed in the Project Charter. Testing focused on correct tool use, missing parameters, unauthorized access, unavailable services and the approval requirement for submitting a support request.

2. Tasks Completed

2.1. Definition of the Tool Catalogue

Four tools were defined. Each has a purpose, an input schema, an output schema, an authorization rule, a side-effect level and documented failure behaviour. The schemas are stored as JSON files in `docs/tool-schemas/`.

Tool	Purpose	Access	Side effect
`lookup_lecture_venue`	Returns the venue and time slots for a course from the timetable	Public	Read only
`get_support_request_status`	Returns the status of a support request the student created	Signed-in owner	Read only
`create_support_request_draft`	Saves a draft support request; nothing is sent to staff	Signed-in student	Low-risk write
`submit_support_request`	Sends a draft to CoCIS support staff	Signed-in owner	Higher-impact write, approval required

Creating a draft and sending a request to staff were defined as separate tools because submission has consequences beyond preparing the request.

Outcome: Each tool has a written contract that the application enforces. The model is given only the tool name, description and input schema.

2.2. Function Calling Through a Deterministic Executor

Gemini automatic function calling was disabled so that the SDK does not run tools by itself. The model can only request a call; the application's executor decides whether it runs. Every request passes through

the same checks, in order: the tool exists, the arguments match the input schema, the caller is signed in and allowed to access the record, approval has been granted where the tool requires it, the handler runs, and the output matches the output schema. Each request is then written to an audit log.

Several design decisions support this process:

- The student's identity comes from the application session and is never accepted as a tool argument.
- Read operations are retried once if the service is unavailable. Write operations are never retried automatically, which avoids duplicate notifications to staff.
- Creating the same draft twice returns the existing draft and marks it as a duplicate.
- The model receives only an error code and a plain message. Stack traces and internal details are hidden, and output that fails its schema is discarded.
- The tool-use loop is limited to five steps.

Outcome: Tool requests from the model are validated, authorized and logged by deterministic code before anything happens.

2.3. Human Approval for Submission

Submitting a support request requires explicit approval. An approval is single-use, is tied to the student, the tool and the exact arguments, and expires after 300 seconds, so an approval given for one action cannot be reused for another. When approval is needed, the workflow pauses and the student is shown which request will be sent and its summary. If approval is denied, nothing is submitted and the request remains a draft.

Outcome: In the demonstration, one approved submission created a single notification for staff, while a second request that was denied created none.

2.4. Demonstration and Failure Testing

The demonstration script `scripts/demo_week4.py` runs five scenarios with a scripted model: a lecture-venue lookup, creating a draft and submitting it with approval, checking the request status, and creating a second draft whose submission is denied. It then sends deliberately faulty requests directly to the executor.

Test	Result
Missing required parameter (`category`)	`INVALID_ARGUMENTS`; the tool did not run and the model was told to ask the student
Student not signed in	`NOT_AUTHENTICATED`
Another student's support request	`NOT_AUTHORIZED`; nothing about the record was revealed
Extra `student_id` argument supplied	`INVALID_ARGUMENTS`; the extra field was rejected
Submission without approval	`APPROVAL_REQUIRED`; nothing was sent

Test	Result
Database unavailable during a read	`SERVICE_UNAVAILABLE`, marked as retryable

At the end of the run, the staff outbox contained exactly one notification, for the approved request. All 13 tool requests, including the failed ones, were recorded in `evidence/week_4/audit-log.jsonl`.

Outcome: The tests showed that invalid, unauthorized and unapproved requests are stopped by the executor, and that failures are reported without exposing internal details or leaving unintended changes.

3. Conclusion

During Week 4, CampusConnect progressed from a system that answers questions to one that can use defined tools under application control. Four tools were specified and implemented, function calling was routed through an executor that validates, authorizes and logs every request, and submission to staff was placed behind explicit approval.

Two limitations remain. Sign-in uses a demonstration identity rather than real university authentication, and the failure tests were run through the executor with a scripted model. The agent workflow built on these tools, and its live execution traces, were developed in Week 5.